

Библиотека NumPy

NumPy (Numeric Python) е библиотека в Python, която се използва за работа с масиви. Съдържа огромно количество функционалности от линейната алгебра, трансформации на Фурие и матрици. NumPy използваме, тъй като голяма част от функционалностите на библиотеката, изискващи голяма изчислителна мощност, са написани на езици от по-ниско ниво на абстракция (главно C).

Елементите на NumPy масивите се съхраняват в последователна памет за разлика от Python list. Това е и една от главните причини NumPy масивите да са значително по-бързи, заедно с оптимизираното изпълнение върху модерни компютърни архитектури и хомогенността на данните, с които работим.

Инсталиране и добавяне на NumPy в проект

Инсталацията на NumPy в текущата ни виртуална среда, както и на много други пакети може да стане с Python инсталатора на пакети pip. Командата пишем в терминала на PyCharm или алтернативата, която използваме.

```
pip install numpy
```

За да добавим библиотеката в проекта си е нужно в началото на кода, който ще я използва да напишем

```
import numpy as np
```

Където "as np" използваме за да създадем псевдоним на библиотеката. Неписано правило в Python е да включваме NumPy с псевдоним np.

Създаване на NumPy масив

```
In [1]: import numpy as np

my_Numpy_array = np.array([1, 2, 3, 4, 5])

my_Numpy_2d_array = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print(my_Numpy_array)
print(type(my_Numpy_array))

print(my_Numpy_2d_array)
print(type(my_Numpy_2d_array))
```

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
[[1 2 3]
 [4 5 6]
 [7 8 9]]
<class 'numpy.ndarray'>
```

Размерности на масиви

В различни сфери се налага работа с многомерни масиви (Пр: компютърна графика, машинно обучение).

```
In [2]: import numpy as np

# 0-D array

arr_0d = np.array(42)

arr_1d = np.array([1, 2, 3, 4, 5, 6])

arr_2d = np.array([[1, 2, 3],[4, 5, 6]])

arr_3d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])

print(f'Array: {arr_0d}, dimensions: {arr_0d.ndim}, shape: {arr_0d.shape}')
print(f'Array: {arr_1d}, dimensions: {arr_1d.ndim}, shape: {arr_1d.shape}')
print(f'Array: {arr_2d}, dimensions: {arr_2d.ndim}, shape: {arr_2d.shape}')
print(f'Array: {arr_3d}, dimensions: {arr_3d.ndim}, shape: {arr_3d.shape}')

Array: 42, dimensions: 0, shape: ()
Array: [1 2 3 4 5 6], dimensions: 1, shape: (6,)
Array: [[1 2 3]
 [4 5 6]], dimensions: 2, shape: (2, 3)
Array: [[[1 2 3]
 [4 5 6]]]
        [[1 2 3]
 [4 5 6]]], dimensions: 3, shape: (2, 2, 3)
```

В примера видяхме:

- `.ndim` - Показва number of dimensions - брой измерения, които има масива
- `.shape` - Показва размерността на масива

Индексиране и слайсване на NumPy масиви

Индексирането и слайсването на NumPy масиви е идентично с това на другите подобни колекции в Python (листи и tuples).

```
In [3]: import numpy as np

my_arr = np.array([[1, 2, 3], [4, 5, 6]])

print(my_arr[0])

print(my_arr[0][1])

print(my_arr[0][:2])

[1 2 3]
2
[1 2]
```

Типове данни в NumPy

По подразбиране в Python имаме следните типове данни:

- string
- integer
- float
- boolean
- complex (За комплексни/имагинерни числа)

В NumPy съществуват допълнителни типове данни, които записваме в "съкратен" вид:

- i - integer
- b - boolean
- u - unsigned integer
- f - float
- c - complex float
- m - timedelta
- M - datetime
- O - object
- S - string
- U - unicode string
- V - фиксирано адресно пространство от друг тип

Типът данни на даден NumPy масив можем да проверим с `.dtype`

```
In [4]: import numpy as np

my_int_arr = np.array([1, 2, 3])

print(my_int_arr.dtype)

my_str_arr = np.array(['John', 'Phil', 'Ben'])

print(my_str_arr.dtype)
```

```
int32  
<U4
```

Можем да създадем масив с точно определен тип данни.

```
In [5]: import numpy as np  
  
my_arr = np.array([1, 2, 3], dtype='S')  
  
print(my_arr)  
print(my_arr.dtype)  
  
[b'1' b'2' b'3']  
|S1
```

Нещо, което може би вече е станало очевидно е, че някои типове в NumPy се дефинират с размерността си. Това са типовете integer, unsigned integer, float, string и unicode string (i, u, f, S, U).

```
In [6]: my_arr = np.array([1, 2, 3], dtype='i1')  
  
print(my_arr)  
print(my_arr.dtype)  
  
[1 2 3]  
int8
```

Конвертиране на типове

Възможно е да направим конвертиране на типове в дефинирането на един NumPy масив:

```
arr = np.array([1.3, 2, 2.1], dtype='i')
```

Това, обаче не е най-чистият начин за конвертиране. Възможно е да получим и грешка (ValueError) ако някой от елементите не могат да бъдат конвертирани.

За промяна на типа данни на съществуващ масив създаваме нов, резултатен масив с .astype() метода. Той създава копие на масива и позволява да посочим типа данни като параметър:

```
In [7]: import numpy as np  
  
arr = np.array([1.2, 2.2, 3.3])  
  
int_arr = arr.astype('i1')  
  
print(int_arr)  
print(int_arr.dtype)
```

```
[1 2 3]
int8
```

Reshape

Промяната на размерността на масиви в NumPy е операция, която често се налага. При работа с матрици, вектори и скалари е важно постоянно да знаем какви са размерностите на нашите данни. `reshape()` метода може да използваме за всякакво преоразмеряване, стига да може масивът ни да пасне на подадената размерност.

```
In [8]: import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])

newarr1 = arr.reshape(6, 2)
print('newarr1: ')
print(newarr1)
print(newarr1.shape)

newarr2 = arr.reshape(2, 6)
print('newarr2: ')
print(newarr2)
print(newarr2.shape)

newarr3 = arr.reshape(4, 3)
print('newarr3: ')
print(newarr3)
print(newarr3.shape)

newarr4 = arr.reshape(3, 2, 2)
print('newarr4: ')
print(newarr4)
print(newarr4.shape)
```

```

newarr1:
[[ 1  2]
 [ 3  4]
 [ 5  6]
 [ 7  8]
 [ 9 10]
 [11 12]]
(6, 2)
newarr2:
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]]
(2, 6)
newarr3:
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
(4, 3)
newarr4:
[[[ 1  2]
   [ 3  4]]

 [[ 5  6]
   [ 7  8]]

 [[ 9 10]
   [11 12]]]
(3, 2, 2)

```

Други полезни методи в NumPy

NumPy е библиотека с огромни възможности и често при работата с нея се обръщаме към документация (<https://numpy.org/doc/>). Някои от другите много често ползвани функционалности са:

- `.zeros()` - Създава масив от нули с даден shape
- `.ones()` - Създава масив от единици с даден shape
- `.eye()` - Създава единична матрица с даден размер. Името идва от `pun` с `identity matrix` - `eye`.
- `.where()` - Намира индексите на елементите, отговарящи на дадена стойност
- `.sort()` - Сортира масива
- "Филтриране" на елементи в даден масив - взима някои елементи на даден масив, копира ги и създава нов масив от тях.

```

In [9]: import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

a = [True, True, False, False, True, False]

filtered_arr = arr[a]
print(filtered_arr)

```

Самостоятелни задачи

1. Създайте програма, която тества разликата между векторизирана операция върху NumPy масив и алтернативата с лист и обхождането му.
2. Създайте функция, която създава boolean матрица от подадена двоична такава.
3. Създайте функция, която проверява дали имаме нулеви елементи в даден NumPy масив. (Разгледайте `.all()` метода)
4. Създайте програма, която извежда всички числа между 4 и 44 от даден масив.
5. Създайте програма, която създава нулева матрица 10x10 с диагонал числата от 1 до 10.